

Lecture 37

STAT 109: Introductory Biostatistics - Simple logistic regression

Lecture 37

From normal to binomial: a simple logistic regression model

For a binary outcome (0/1, No/Yes), we replace normal-errors linear regression with a binomial model:

$$\text{logit}(p) = \beta_0 + \beta_1 x, \quad p = P(Y = 1|X = x)$$

Fit in R with:

- `glm(y ~ x, family = binomial, data = ...)`

Main prediction target is probability p , then classify using a cutoff (often 0.5 unless context suggests otherwise).

Learning objectives

By the end of this lecture, you should be able to:

- explain why logistic regression is used for binary outcomes;
 - fit and interpret a simple logistic regression model with `glm(..., family = binomial)`;
 - interpret the slope as a change in log-odds (and direction of odds change);
 - compute and interpret a confidence interval for the logistic slope;
 - generate predicted probabilities and classify outcomes using a cutoff;
 - compare training and test accuracy and interpret out-of-sample classification performance.
-

Example 1 - Study hours and pass/fail

```
if (!requireNamespace("tidyverse", quietly = TRUE)) {  
  install.packages("tidyverse", repos = "https://cloud.r-project.org")  
}  
  
library(tidyverse)  
  
set.seed(3601)  
n <- 160  
pass_dat <- tibble(  
  study_hours = runif(n, 0, 15)  
) |>  
  mutate(  
    p_pass = plogis(-2.4 + 0.38 * study_hours),  
    pass = rbinom(n(), 1, p_pass)  
)
```

Train/test split and fit

```
set.seed(3602)
idx <- sample(seq_len(nrow(pass_dat)), size = 0.8 * nrow(pass_dat))
pass_train <- pass_dat[idx, ]
pass_test  <- pass_dat[-idx, ]

fit_pass <- glm(pass ~ study_hours, family = binomial, data = pass_train)
summary(fit_pass)
```

Call:

```
glm(formula = pass ~ study_hours, family = binomial, data = pass_train)
```

Coefficients:

```
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.91993     0.57498  -5.078 3.81e-07 ***
study_hours  0.44040     0.07299   6.034 1.60e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 172.92  on 127  degrees of freedom
Residual deviance: 109.15  on 126  degrees of freedom
AIC: 113.15
```

Number of Fisher Scoring iterations: 5

CI for slope beta1

```
confint(fit_pass, "study_hours")
```

```
      2.5 %      97.5 %
0.3092083 0.5978433
```

If the CI is entirely above 0, more study hours are associated with higher odds of passing.

Predicted probability and cutoff-based class prediction

```
pass_train <- pass_train |>
  mutate(
    p_hat = predict(fit_pass, newdata = pass_train, type = "response"),
    y_hat_50 = if_else(p_hat >= 0.5, 1, 0)
  )

pass_test <- pass_test |>
  mutate(
    p_hat = predict(fit_pass, newdata = pass_test, type = "response"),
    y_hat_50 = if_else(p_hat >= 0.5, 1, 0)
  )
```

Accuracy on training and test

```
acc <- function(y, yhat) mean(y == yhat)

tibble(
  set = c("train", "test"),
  accuracy = c(
    acc(pass_train$pass, pass_train$y_hat_50),
    acc(pass_test$pass, pass_test$y_hat_50)
  )
)
```

```
# A tibble: 2 x 2
  set    accuracy
<chr>   <dbl>
1 train  0.844
2 test   0.844
```

Training accuracy can be optimistic; test accuracy better represents performance on new data.

Probability at new x and different cutoffs

```
new_students <- tibble(study_hours = c(4, 8, 12))
p_new <- predict(fit_pass, newdata = new_students, type = "response")
tibble(study_hours = new_students$study_hours, predicted_probability_pass = p_new)
```

```
# A tibble: 3 x 2
  study_hours predicted_probability_pass
      <dbl>             <dbl>
1         4             0.239
2         8             0.646
3        12             0.914
```

```
cutoff_demo <- tibble(
  cutoff = c(0.3, 0.5, 0.7),
  predicted_class_for_p0_62 = if_else(0.62 >= cutoff, 1, 0)
)
cutoff_demo
```

```
# A tibble: 3 x 2
  cutoff predicted_class_for_p0_62
    <dbl>             <dbl>
1    0.3             1
2    0.5             1
3    0.7             0
```

Example 2 - Fertilizer dose and bloom/no-bloom

```
set.seed(3603)
bloom_dat <- tibble(
  dose = runif(140, 0, 12)
```

```

) |>
  mutate(
    p_bloom = plogis(-1.9 + 0.42 * dose),
    bloom = rbinom(n(), 1, p_bloom)
  )

idx <- sample(seq_len(nrow(bloom_dat)), size = 0.8 * nrow(bloom_dat))
bloom_train <- bloom_dat[idx, ]
bloom_test  <- bloom_dat[-idx, ]

fit_bloom <- glm(bloom ~ dose, family = binomial, data = bloom_train)
summary(fit_bloom)

```

Call:

```
glm(formula = bloom ~ dose, family = binomial, data = bloom_train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-2.28728	0.49638	-4.608	4.07e-06 ***
dose	0.50458	0.09239	5.462	4.72e-08 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 152.97 on 111 degrees of freedom
 Residual deviance: 101.94 on 110 degrees of freedom
 AIC: 105.94

Number of Fisher Scoring iterations: 5

```
confint(fit_bloom, "dose")
```

	2.5 %	97.5 %
dose	0.3392949	0.7051015

```

bloom_test <- bloom_test |>
  mutate(
    p_hat = predict(fit_bloom, newdata = bloom_test, type = "response"),
    y_hat_50 = if_else(p_hat >= 0.5, 1, 0)
  )

tibble(test_accuracy = mean(bloom_test$bloom == bloom_test$y_hat_50))

```

```

# A tibble: 1 x 1
  test_accuracy
      <dbl>
1         0.75

```

Example 3 - Exercise minutes and hypertension yes/no

```
set.seed(3604)
bp_dat <- tibble(
  exercise_min = runif(170, 0, 70)
) |>
  mutate(
    p_hyp = plogis(0.9 - 0.045 * exercise_min),
    hypertension = rbinom(n(), 1, p_hyp)
  )

idx <- sample(seq_len(nrow(bp_dat)), size = 0.8 * nrow(bp_dat))
bp_train <- bp_dat[idx, ]
bp_test <- bp_dat[-idx, ]

fit_bp <- glm(hypertension ~ exercise_min, family = binomial, data = bp_train)
summary(fit_bp)
```

Call:

```
glm(formula = hypertension ~ exercise_min, family = binomial,
    data = bp_train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.99507	0.36133	2.754	0.00589 **
exercise_min	-0.04804	0.01033	-4.651	3.3e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 178.89 on 135 degrees of freedom
Residual deviance: 152.25 on 134 degrees of freedom
AIC: 156.25

Number of Fisher Scoring iterations: 4

```
confint(fit_bp, "exercise_min")
```

	2.5 %	97.5 %
exercise_min	-0.06947402	-0.02873974

```
bp_test <- bp_test |>
  mutate(
    p_hat = predict(fit_bp, newdata = bp_test, type = "response"),
    y_hat_50 = if_else(p_hat >= 0.5, 1, 0)
  )

tibble(
  test_accuracy = mean(bp_test$hypertension == bp_test$y_hat_50),
  p_at_20min = predict(fit_bp, newdata = tibble(exercise_min = 20), type = "response"),
```

```
p_at_50min = predict(fit_bp, newdata = tibble(exercise_min = 50), type = "response")
)
```

```
# A tibble: 1 x 3
  test_accuracy p_at_20min p_at_50min
      <dbl>      <dbl>      <dbl>
1      0.794      0.509      0.197
```

Key takeaways

- Logistic regression models probability for a binary outcome via the logit link.
- Use training data to fit the model; report test accuracy for generalization.
- Slope CI tells whether the log-odds effect is plausibly positive, negative, or near zero.
- Probability prediction comes first; class prediction depends on chosen cutoff.